

The Masterclass Guide to CBOW

A Deep, Intuitive, Step-by-Step Mathematical Walkthrough

Version 2.0 • Expanded for Absolute Clarity, Fully Worked Derivations, and Matrix Verification

1 Introduction: The Conceptual Foundation & Word Duality

The **Continuous Bag-of-Words (CBOW)** model is one of the foundational algorithms of modern Natural Language Processing. Its objective is highly intuitive: **predict a center target word based on its surrounding context words**.

However, students often get stuck trying to follow the mathematics of traditional papers because of abstract notation (W vs. W') and skipped steps in backpropagation. This guide is designed to fix that by building the intuition from the ground up.

The Core Intuition: The Two-Vector Duality

To understand CBOW, you must first accept a counter-intuitive fact: **every word in our vocabulary possesses TWO completely distinct vector identities**.

Why Two Vectors per Word?

Imagine a word acts differently depending on its role in a sentence:

1. **When acting as a Context Clue ($\mathbf{v}$):** The word is part of the surrounding environment. It casts a “vote” or exerts a “pull” toward what the center word should be.
2. **When acting as the Target Candidate ($\mathbf{v}'$):** The word is the candidate being tested. We measure how well its properties align with the collective pull of the context clues.

By keeping these two roles separate, the math becomes perfectly clean. At the end of training, we typically use the context vectors ($\mathbf{v}$) as our final word embeddings, though sometimes the two are averaged.

Renaming the Matrices for Absolute Clarity

Instead of the confusing symbols W and W' , we define two highly descriptive weight matrices that the neural network learns:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{V \times N}$ is the **Input Embedding Matrix**. Row k contains the context vector $\mathbf{v}_k$ for word k .
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{N \times V}$ is the **Output Embedding Matrix**. Column k contains the target vector $\mathbf{v}'_k$ for word k .

Global Dimensions Used Throughout This Guide:

- V : **Vocabulary size** (total unique words in the language).
- N : **Embedding dimension** (the number of features in our hidden space).
- C : **Context window size** (total number of surrounding words used to guess the target).

2 The Forward Pass: Tracing the Matrix Pipeline

Let's trace exactly how a set of context words transforms into a probability distribution over the vocabulary. We will explicitly track matrix dimensions at every single step so you never get lost.

Step 1: One-Hot Context Vectors ($\mathbf{x}$)

For each surrounding context word c_i (where $i \in \{1, \dots, C\}$), we represent it as a sparse one-hot column vector:

$$\mathbf{x}_{c_i} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^{V \times 1} \quad (\text{contains a single 1 at index } c_i, \text{ and 0 elsewhere})$$

Step 2: Projection / Lookup Layer ($\mathbf{v}$)

We project each context word from the sparse vocabulary space into the dense hidden embedding space by multiplying the transposed input matrix by the one-hot vector:

$$\underbrace{\mathbf{v}_{c_i}}_{N \times 1} = \underbrace{\mathbf{W}_{\text{in}}^\top}_{N \times V} \times \underbrace{\mathbf{x}_{c_i}}_{V \times 1}$$

Visualizing the Lookup: Matrix multiplication with a one-hot vector involves no actual arithmetic; it is a pure **indexing operation** that instantly plucks out the c_i -th row of $\mathbf{W}_{\text{in}}$:

$$\mathbf{W}_{\text{in}}^\top \mathbf{x}_{c_i} = \begin{bmatrix} | & | & \dots & | & \dots & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_{c_i} & \dots & \mathbf{v}_V \\ | & | & & | & & | \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} = \mathbf{v}_{c_i}$$

Step 3: The “Bag” Averaging (Forming the Centroid $\mathbf{h}$)

We collapse all C individual context vectors into a single collective hidden state $\mathbf{h}$ by averaging them:

$$\underbrace{\mathbf{h}}_{N \times 1} = \frac{1}{C} \sum_{i=1}^C \mathbf{v}_{c_i} = \frac{1}{C} \mathbf{W}_{\text{in}}^\top \left(\sum_{i=1}^C \mathbf{x}_{c_i} \right)$$

Geometric Intuition: The vector $\mathbf{h}$ represents the **geometric centroid** (center point) of the context clues in the N -dimensional space. Because addition is commutative, word order is entirely discarded—hence the term “Bag of Words.”

Step 4: Scoring All Vocabulary Candidates ($\mathbf{u}$)

We now measure how closely aligned our context centroid $\mathbf{h}$ is to every possible target word's output representation. We compute raw similarity scores (logits) $\mathbf{u}$:

$$\underbrace{\mathbf{u}}_{V \times 1} = \underbrace{\mathbf{W}_{\text{out}}^{\top}}_{V \times N} \times \underbrace{\mathbf{h}}_{N \times 1}$$

Each component $u_j = \mathbf{v}'_j \cdot \mathbf{h}$ is a pure **dot product**. If the output vector $\mathbf{v}'_j$ points in the exact same direction as the context centroid $\mathbf{h}$, the score u_j is highly positive. If they are orthogonal, the score is zero.

Step 5: Softmax Normalization ($\mathbf{y}$)

Raw scores $\mathbf{u}$ can be any real number. To convert them into valid probabilities that sum to 1, we pass them through the Softmax function:

$$y_j = P(w_j \mid \text{context}) = \frac{\exp(u_j)}{\sum_{k=1}^V \exp(u_k)}$$

The Complete Forward Dimension Flow

Notice how elegantly the dimensions expand and contract:

$$\underbrace{\mathbf{x}_{c_i}}_{V \times 1} \xrightarrow{\text{Lookup}} \underbrace{\mathbf{v}_{c_i}}_{N \times 1} \xrightarrow{\text{Average}} \underbrace{\mathbf{h}}_{N \times 1} \xrightarrow{\text{Dot Product}} \underbrace{\mathbf{u}}_{V \times 1} \xrightarrow{\text{Softmax}} \underbrace{\mathbf{y}}_{V \times 1}$$

3 Fully Worked Numerical Example

Let's ground these abstract formulas in concrete arithmetic using a tiny vocabulary. Watching the actual numbers flow makes the mechanics unmistakable.

Setup & Parameters

- **Corpus Sentence:** "the cat sat on mat"
- **Vocabulary** ($V = 5$): the=0, cat=1, sat=2, on=3, mat=4
- **Hyperparameters:** Hidden dimension $N = 3$, Context window $C = 2$.
- **Current Training Sample:** Context words = {cat, on}, True Target word = **sat** (Index $t = 2$).

Initial Small Random Weight Matrices

Let the initial **Input Matrix** $\mathbf{W}_{\text{in}}$ (5×3) be:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ \mathbf{0.4} & \mathbf{0.5} & \mathbf{0.6} \\ 0.7 & 0.8 & 0.9 \\ \mathbf{0.2} & \mathbf{0.1} & \mathbf{0.4} \\ 0.5 & 0.3 & 0.2 \end{bmatrix} \begin{array}{l} \text{the} \\ \text{cat} \leftarrow \text{Context clue } c_1 \\ \text{sat} \\ \text{on} \leftarrow \text{Context clue } c_2 \\ \text{mat} \end{array}$$

Let the initial **Output Matrix** $\mathbf{W}_{\text{out}}$ (3×5) be:

$$\mathbf{W}_{\text{out}} = \begin{bmatrix} 0.1 & 0.3 & \mathbf{0.5} & 0.2 & 0.4 \\ 0.2 & 0.1 & \mathbf{0.6} & 0.3 & 0.5 \\ 0.4 & 0.2 & \mathbf{0.7} & 0.1 & 0.3 \end{bmatrix} \begin{array}{l} \uparrow \\ \text{sat (True Target Column } t = 2) \end{array}$$

Step-by-Step Numerical Execution

1. Extract Context Vectors (Row Lookups): From rows 1 and 3 of $\mathbf{W}_{\text{in}}$, we immediately read off:

$$\mathbf{v}_{\text{cat}} = \begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix}, \quad \mathbf{v}_{\text{on}} = \begin{bmatrix} 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}$$

2. Compute the Centroid $\mathbf{h}$:

$$\mathbf{h} = \frac{\mathbf{v}_{\text{cat}} + \mathbf{v}_{\text{on}}}{2} = \frac{1}{2} \left(\begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix} + \begin{bmatrix} 0.2 \\ 0.1 \\ 0.4 \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} 0.6 \\ 0.6 \\ 1.0 \end{bmatrix} = \begin{bmatrix} 0.3 \\ 0.3 \\ 0.5 \end{bmatrix}$$

3. Compute Candidate Scores $\mathbf{u} = \mathbf{W}_{\text{out}}^\top \mathbf{h}$: We take the dot product of our centroid $\mathbf{h}$ with

every single column of $\mathbf{W}_{\text{out}}$:

$$\begin{aligned} u_{\text{the}} &= (0.1)(0.3) + (0.2)(0.3) + (0.4)(0.5) = 0.03 + 0.06 + 0.20 = \mathbf{0.29} \\ u_{\text{cat}} &= (0.3)(0.3) + (0.1)(0.3) + (0.2)(0.5) = 0.09 + 0.03 + 0.10 = \mathbf{0.22} \\ u_{\text{sat}} &= (\mathbf{0.5})(\mathbf{0.3}) + (\mathbf{0.6})(\mathbf{0.3}) + (\mathbf{0.7})(\mathbf{0.5}) = \mathbf{0.15} + \mathbf{0.18} + \mathbf{0.35} = \mathbf{0.68} \\ u_{\text{on}} &= (0.2)(0.3) + (0.3)(0.3) + (0.1)(0.5) = 0.06 + 0.09 + 0.05 = \mathbf{0.20} \\ u_{\text{mat}} &= (0.4)(0.3) + (0.5)(0.3) + (0.3)(0.5) = 0.12 + 0.15 + 0.15 = \mathbf{0.42} \end{aligned}$$

Resulting score vector: $\mathbf{u} = [0.29, 0.22, 0.68, 0.20, 0.42]^\top$.

4. Softmax Probabilities $\mathbf{y}$: First, exponentiate the scores: $\exp(\mathbf{u}) \approx [1.336, 1.246, 1.974, 1.221, 1.522]^\top$. Next, sum these values: $1.336 + 1.246 + 1.974 + 1.221 + 1.522 \approx \mathbf{7.299}$. Finally, divide each exponentiated score by the sum to get probabilities:

$$\mathbf{y} \approx \begin{bmatrix} 1.336/7.299 \\ 1.246/7.299 \\ 1.974/7.299 \\ 1.221/7.299 \\ 1.522/7.299 \end{bmatrix} \approx \begin{bmatrix} 0.183 \\ 0.171 \\ \mathbf{0.270} \\ 0.167 \\ 0.209 \end{bmatrix} \begin{matrix} \text{the} \\ \text{cat} \\ \text{sat} \leftarrow \text{True target!} \\ \text{on} \\ \text{mat} \end{matrix}$$

Current Status: The untrained model assigns a **27.0% probability** to the correct target **sat**. Since random guessing over 5 words would yield 20%, it's slightly better than chance, but training needs to drive this much higher.

5. Compute the Loss: We use the Cross-Entropy loss $\mathcal{L}$ on the true target word t :

$$\mathcal{L} = -\log(y_t) = -\log(0.270) \approx \mathbf{1.309}$$

4 Demystifying Backpropagation: Zero Skipped Steps

Training the network requires taking the derivative of the loss $\mathcal{L}$ with respect to our weights. Let's walk through the exact mathematical calculus slowly.

Deriving the Fundamental Error Vector ($\mathbf{e} = \partial\mathcal{L}/\partial\mathbf{u}$)

This is the single most important derivation in Word2Vec. We want to know how changing each raw score u_j affects the total loss $\mathcal{L}$. Let's expand the loss formula fully:

$$\mathcal{L} = -\log(y_t) = -\log\left(\frac{\exp(u_t)}{\sum_{k=1}^V \exp(u_k)}\right) = -u_t + \log\left(\sum_{k=1}^V \exp(u_k)\right)$$

We want to compute the partial derivative $\frac{\partial\mathcal{L}}{\partial u_j}$ for any arbitrary word index j . Let's differentiate term by term:

$$\frac{\partial\mathcal{L}}{\partial u_j} = \underbrace{\frac{\partial}{\partial u_j}(-u_t)}_{\text{Term A}} + \underbrace{\frac{\partial}{\partial u_j} \log\left(\sum_{k=1}^V \exp(u_k)\right)}_{\text{Term B}}$$

Evaluating Term A: The term $-u_t$ only depends on u_j if the index j happens to be the true target t . Otherwise, the derivative is zero:

$$\text{Term A} = \begin{cases} -1 & \text{if } j = t \\ 0 & \text{if } j \neq t \end{cases} \equiv -\mathbb{I}[j = t]$$

Evaluating Term B (Using the Chain Rule): Let $S = \sum_{k=1}^V \exp(u_k)$. The outer derivative of $\log(S)$ is $\frac{1}{S}$. The inner derivative $\frac{\partial S}{\partial u_j}$ only picks up the single term where $k = j$, which is $\exp(u_j)$:

$$\text{Term B} = \frac{1}{S} \cdot \frac{\partial S}{\partial u_j} = \frac{1}{\sum_{k=1}^V \exp(u_k)} \cdot \exp(u_j) = \frac{\exp(u_j)}{\sum_{k=1}^V \exp(u_k)} = y_j$$

Notice the beautiful magic: the derivative of the log-sum-exp term is **exactly the softmax probability** y_j !

Combining Terms A and B:

$$e_j \equiv \frac{\partial\mathcal{L}}{\partial u_j} = y_j - \mathbb{I}[j = t]$$

The Elegant Vector Result: Prediction minus Truth

In vector form, the error flowing out of the scores is incredibly simple:

$$\mathbf{e} = \mathbf{y} - \mathbf{y}_{\text{true}}$$

Where $\mathbf{y}_{\text{true}}$ is the one-hot vector of the true target. Let's compute $\mathbf{e}$ for our numerical example ($t = 2$):

$$\mathbf{e} \approx \begin{bmatrix} 0.183 \\ 0.171 \\ 0.270 \\ 0.167 \\ 0.209 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.183 \\ 0.171 \\ -0.730 \\ 0.167 \\ 0.209 \end{bmatrix}$$

Crucial Intuition: The error for the true target **sat** is **negative** (-0.730), signaling the weights to pull its score UP. The errors for all incorrect words are **positive**, signaling the weights to push their scores DOWN.

5 Gradients for the Weights & The Outer Product

Now that we have the score error $\mathbf{e}$, let's propagate it backward to the weight matrices.

Gradient w.r.t. Output Matrix ($\mathbf{W}_{\text{out}}$)

Recall our forward equation: $\mathbf{u} = \mathbf{W}_{\text{out}}^\top \mathbf{h}$. How do we get the derivative w.r.t. the matrix?

The Golden Shortcut for Linear Layers

For any dense layer $\mathbf{output} = \mathbf{Matrix}^\top \cdot \mathbf{input}$, the gradient w.r.t. the matrix is always the **outer product** of the incoming input vector and the outgoing error vector:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Matrix}} = \mathbf{input} \times \mathbf{error}^\top$$

Plugging in our variables ($\mathbf{input} = \mathbf{h}$ and $\mathbf{error} = \mathbf{e}$), we immediately get:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{out}}} = \underbrace{\mathbf{h}}_{N \times 1} \times \underbrace{\mathbf{e}^\top}_{1 \times V} \in \mathbb{R}^{N \times V}$$

Why an Outer Product Makes Complete Physical Sense: Think about credit assignment. A specific weight $W_{\text{out}}^{(m,j)}$ connects hidden unit h_m to the score u_j . It deserves a large correction only if TWO conditions are met:

1. The hidden unit h_m was highly active (high input responsibility).
2. The predicted score u_j was highly incorrect (high output error e_j).

Multiplying them together ($h_m \cdot e_j$) perfectly captures this simultaneous condition!

Gradient w.r.t. the Hidden Centroid ($\mathbf{h}$)

Using the standard chain rule, the error flowing backward into the hidden state $\mathbf{h}$ is a weighted sum of the rows of $\mathbf{W}_{\text{out}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \underbrace{\mathbf{W}_{\text{out}}}_{N \times V} \times \underbrace{\mathbf{e}}_{V \times 1} \in \mathbb{R}^{N \times 1}$$

Gradient w.r.t. the Input Matrix ($\mathbf{W}_{\text{in}}$)

Recall that $\mathbf{h} = \frac{1}{C} \sum_{i=1}^C \mathbf{v}_{c_i}$. The gradient distributes equally back to the specific rows of $\mathbf{W}_{\text{in}}$ corresponding to the C context words:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{in}}^{(c_i,:)}} = \frac{1}{C} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{h}} \right)^\top = \frac{1}{C} (\mathbf{W}_{\text{out}} \mathbf{e})^\top \quad \text{for each context word } c_i$$

Massive Sparsity Advantage: All other rows in $\mathbf{W}_{\text{in}}$ (corresponding to words not in the current context window) receive exactly **zero gradient**. This sparsity is why Word2Vec trains incredibly fast over vocabulary sizes of millions!

Summary Table of All Gradients

Target Quantity	Formula	Matrix Shape	Computational Meaning
Score Error $\mathbf{e}$	$\mathbf{y} - \mathbf{y}_{\text{true}}$	$V \times 1$	Subtraction (Prediction minus Truth)
Output Matrix $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{out}}}$	$\mathbf{h}\mathbf{e}^\top$	$N \times V$	Outer Product
Hidden State $\frac{\partial \mathcal{L}}{\partial \mathbf{h}}$	$\mathbf{W}_{\text{out}}\mathbf{e}$	$N \times 1$	Matrix-Vector Multiplication
Input Rows $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{in}}^{(c_i,:)}}$	$\frac{1}{C}(\mathbf{W}_{\text{out}}\mathbf{e})^\top$	$1 \times N$	Sparse Row Scatter (C rows only)

6 Complete Numerical Verification (Proof That It Works)

Let's perform one actual step of Stochastic Gradient Descent (SGD) with a learning rate $\eta = 0.1$ on our numerical example and prove that the network successfully learns.

1. Updating the Output Matrix Column for **sat**

The update rule for the target column $t = 2$ (**sat**) is:

$$\mathbf{v}'_{\text{sat}} \leftarrow \mathbf{v}'_{\text{sat}} - \eta \cdot (\mathbf{h} \cdot e_{\text{sat}})$$

Let's plug in our known numbers ($\mathbf{h} = [0.3, 0.3, 0.5]^\top$ and $e_{\text{sat}} = -0.730$):

$$\text{Correction} = 0.1 \cdot \begin{bmatrix} 0.3 \\ 0.3 \\ 0.5 \end{bmatrix} \cdot (-0.730) = \begin{bmatrix} -0.0219 \\ -0.0219 \\ -0.0365 \end{bmatrix}$$

Subtracting this correction from the original target column:

$$\mathbf{v}'_{\text{sat (new)}} = \begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \end{bmatrix} - \begin{bmatrix} -0.0219 \\ -0.0219 \\ -0.0365 \end{bmatrix} = \begin{bmatrix} \mathbf{0.5219} \\ \mathbf{0.6219} \\ \mathbf{0.7365} \end{bmatrix}$$

Observation: The components of the target vector increased, moving directly **toward** the positive components of the centroid $\mathbf{h}$.

2. Computing the Hidden Error ($\partial\mathcal{L}/\partial\mathbf{h}$)

Let's multiply $\mathbf{W}_{\text{out}}\mathbf{e}$:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{h}} = \begin{bmatrix} 0.1 & 0.3 & 0.5 & 0.2 & 0.4 \\ 0.2 & 0.1 & 0.6 & 0.3 & 0.5 \\ 0.4 & 0.2 & 0.7 & 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0.183 \\ 0.171 \\ -0.730 \\ 0.167 \\ 0.209 \end{bmatrix} = \begin{bmatrix} 0.0183 + 0.0513 - 0.3650 + 0.0334 + 0.0836 \\ 0.0366 + 0.0171 - 0.4380 + 0.0501 + 0.1045 \\ 0.0732 + 0.0342 - 0.5110 + 0.0167 + 0.0627 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.1784} \\ \mathbf{-0.2297} \\ \mathbf{-0.3242} \end{bmatrix}$$

3. Updating the Input Rows for **cat** and **on**

Each context word receives $\frac{\eta}{C} = \frac{0.1}{2} = 0.05$ times the hidden error:

$$\text{Row Correction} = 0.05 \cdot [-0.1784, -0.2297, -0.3242] = [-0.0089, -0.0115, -0.0162]$$

Subtracting this from the original rows in $\mathbf{W}_{\text{in}}$:

$$\mathbf{v}_{\text{cat (new)}} = [0.4, 0.5, 0.6] - [-0.0089, -0.0115, -0.0162] = [\mathbf{0.4089}, \mathbf{0.5115}, \mathbf{0.6162}]$$

$$\mathbf{v}_{\text{on (new)}} = [0.2, 0.1, 0.4] - [-0.0089, -0.0115, -0.0162] = [\mathbf{0.2089}, \mathbf{0.1115}, \mathbf{0.4162}]$$

4. The Ultimate Proof: Recomputing the Target Score

Let's see if the new centroid and new target vector yield a higher score for the correct word **sat**:

$$\mathbf{h}_{\text{new}} = \frac{\mathbf{v}_{\text{cat (new)}} + \mathbf{v}_{\text{on (new)}}}{2} = \begin{bmatrix} 0.3089 \\ 0.3115 \\ 0.5162 \end{bmatrix}$$

Taking the dot product with the updated target vector $\mathbf{v}'_{\text{sat (new)}}$:

$$u_{\text{sat (new)}} = (0.5219)(0.3089) + (0.6219)(0.3115) + (0.7365)(0.5162) = 0.1612 + 0.1937 + 0.3802 = \mathbf{0.7351}$$

Success!

Our original raw score for **sat** was **0.6800**. After just one single step of matrix backpropagation, the score increased to **0.7351**! The probabilities of the correct word will rise, the loss will drop, and the vectors are successfully learning semantic alignment.